SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence
COURSE CODE: CSA1708

COURSE OUTCOME COVERED

CO3: Analyze knowledge representation and reasoning using propositional logic, FOL, inference, resolution,

chaining methods, knowledge representation to perform logical reasoning for drawing valid conclusions and

decision making. (BL4)

Assessment Tool Weightage: 50%

QUESTIONS: 4 **Total Marks:40**

Annexure A

CASE STUDY

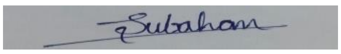
Code of Conduct:

I N.Kmaeshwar reddy192525272) certify that this submission is my original work and

that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

CASE STUDY

Case Study 1 — Smart Medical Diagnosis System

A hospital is designing a rule-based Logical Agent for preliminary patient diagnosis. The system's knowledge

base contains:

I. If a patient has Fever AND Cough → Possible Flu

Ii. If a patient has Fever AND Rash → Possible Measles

Iii. If a patient has Cough AND Breathlessness → Possible Pneumonia

Iv. Facts about Patient A: Fever = True, Cough = True, Breathlessness = True

(a) Represent the above rules and facts precisely using Propositional Logic symbols. Define each symbol used.

(b) Apply Forward Chaining on the knowledge base to derive all possible diagnoses for Patient A. Show each inference step with the rule fired and fact derived.

(c) The goal is to confirm 'Patient A has Pneumonia.' Apply Backward Chaining and draw the goal reduction tree to verify this goal.

(a) Propositional Logic Representation

Let the following propositions represent the patient's symptoms and diagnoses:

- F = Patient has Fever
- C = Patient has Cough
- B = Patient has Breathlessness
- FL = Patient has Possible Flu
- M = Patient has Possible Measles
- P = Patient has Possible Pneumonia

The rules can be represented as:

1. Fever AND Cough → Possible Flu
2. Fever AND Rash → Possible Measles
3. Cough AND Breathlessness → Possible Pneumonia

Facts about Patient A:

There is no fact stating Rash = True, so the measles rule cannot be fired.

(b) Forward Chaining

Forward Chaining starts with the known facts and repeatedly applies rules whose conditions are satisfied.

Initial Facts

Step 1 — Apply Rule I

Rule:

Since:

both conditions are satisfied.

Derived fact: Patient A has Possible Flu.

Step 2 — Apply Rule III

Rule:

Since:

both conditions are satisfied.

Derived fact: Patient A has Possible Pneumonia.

Step 3 — Apply Rule II

Rule:

Although Fever is true, there is no fact that Rash is true.

Therefore, Rule II cannot be fired

Forward Chaining Summary

Step	Rule Fired	Conditions Satisfied	New Fact
0	—	Initial facts	F, C, B
1	Rule I	$F \wedge C$	FL
2	Rule III	$C \wedge B$	P
3	Rule II	$F \wedge R$ — R is false/unknown	No conclusion

(c) Backward Chaining for Pneumonia

The goal is:

where P = Patient A has Possible Pneumonia.

Backward chaining starts with the goal and works backward to determine whether its required conditions are known facts**.**

Step 1 — Start with Goal - P

Find a rule that can conclude P.

Rule III: (C∧B)→P

Therefore, the goal is reduced to: C∧B

Step 2 — Verify Subgoals

Subgoal 1 – C

Known fact - C=True

Therefore: C ✓

Goal Reduction Tree

```
              GOAL

               P

       Patient A has Pneumonia

              |

           Rule III

       Cough AND Breathlessness

          /         \

         /           \

        C             B

     Cough = True    Breathlessness = True

        ✓              ✓

         \             /

          \           /

           \         /

            \       /
```

GOAL PROVED

Using **Forward Chaining**, the system derives:

- Possible Flu
- Possible Pneumonia

Case Study 2 — Autonomous Traffic Management Agent

A smart city traffic controller uses a FOL-based knowledge base to manage emergency vehicle routing:

I. ∀x: Vehicle(x) ∧ EmergencyType(x) → ClearPath(x) [Rule 1]

II. ∀x: ClearPath(x) → GreenSignal(x) ∧ ¬RedSignal(x) [Rule 2]

III. ∀x ∀y: Vehicle(x) ∧ Behind(x,y) ∧ GreenSignal(y) → Proceed(x) [Rule 3]

IV. Facts: Vehicle(Ambulance), EmergencyType(Ambulance), Vehicle(CarA), Behind(CarA,

Ambulance)

(a) Apply Unification to match Rule 1 with the given facts. Show the complete substitution set θ (MGU) at Given Knowledge Base

(b) Using the Resolution method, prove that 'Proceed (CarA)' is true. Convert relevant clauses to CNF and show each resolution step.

(c) Analyze whether the current FOL knowledge base is sufficient to handle two simultaneous emergency vehicles. Identify what additional rules or facts are needed and justify logically.

(a) Unification

Given Rule 1

∀x : Vehicle(x) AND EmergencyType(x) → ClearPath(x)

Given Facts

Vehicle(Ambulance)

EmergencyType(Ambulance)

Vehicle(CarA)

Behind(CarA, Ambulance)

Step 1

Rule 1 requires:

Vehicle(x)

Given fact:

Vehicle(Ambulance)

Therefore:

x = Ambulance

Substitution:

$\theta 1 = \{x/\text{Ambulance}\}$

Step 2

Rule 1 also requires:

EmergencyType(x)

Substitute:

x = Ambulance

We get:

EmergencyType(Ambulance)

Given fact:

EmergencyType(Ambulance)

Therefore:

$\theta 2 = \{x/\text{Ambulance}\}$

Applying the substitution:

Vehicle(Ambulance) AND EmergencyType(Ambulance)

→ ClearPath(Ambulance)

Therefore:

ClearPath(Ambulance)

(b) Resolution Method

Step 1: Convert Rule 1 into CNF

Original Rule 1:

Vehicle(x) AND EmergencyType(x) → ClearPath(x)

CNF:

NOT Vehicle(x) OR NOT EmergencyType(x) OR ClearPath(x)

Step 2: Convert Rule 2 into CNF

Original Rule 2:

ClearPath(x) → GreenSignal(x) AND NOT RedSignal(x)

First clause:

NOT ClearPath(x) OR GreenSignal(x)

Second clause:

NOT ClearPath(x) OR NOT RedSignal(x)

Only the first clause is required for proving Proceed(CarA).

Step 3: Convert Rule 3 into CNF

Original Rule 3:

Vehicle(x) AND Behind(x,y) AND GreenSignal(y) → Proceed(x)

CNF:

NOT Vehicle(x) OR NOT Behind(x,y) OR NOT GreenSignal(y) OR Proceed(x)

Step 4: Given Facts

Vehicle(Ambulance)

EmergencyType(Ambulance)

Vehicle(CarA)

Behind(CarA,Ambulance)

Goal:

Proceed(CarA)

Negated goal:

NOT Proceed(CarA

Step 5: Derive ClearPath(Ambulance)

Rule 1:

NOT Vehicle(x) OR NOT EmergencyType(x) OR ClearPath(x)

Fact:

Vehicle(Ambulance)

Substitution:

x = Ambulance

After resolution:

NOT EmergencyType(Ambulance) OR ClearPath(Ambulance)

Fact:

EmergencyType(Ambulance)

After resolution:

ClearPath(Ambulance)

Step 6: Derive GreenSignal(Ambulance)

Rule 2:

NOT ClearPath(x) OR GreenSignal(x)

Fact:

ClearPath(Ambulance)

Substitution:

x = Ambulance

Therefore:

GreenSignal(Ambulance)

Step 7: Derive Proceed(CarA)

Rule 3:

NOT Vehicle(x) OR NOT Behind(x,y) OR NOT GreenSignal(y) OR Proceed(x)

Fact:

Vehicle(CarA)

Substitution:

x = CarA

After resolution:

NOT Behind(CarA,y) OR NOT GreenSignal(y) OR Proceed(CarA)

Fact:

Behind(CarA,Ambulance)

Substitution:

y = Ambulance

After resolution:

NOT GreenSignal(Ambulance) OR Proceed(CarA)

Fact:

GreenSignal(Ambulance)

After resolution:

Proceed(CarA)

Therefore:

Proceed(CarA) = TRUE

Step 8: Resolution with Negated Goal

Negated goal:

NOT Proceed(CarA)

Derived result:

Proceed(CarA)

Resolve both:

Proceed(CarA)

NOT Proceed(CarA)

Result:

EMPTY CLAUSE

□

Therefore:

Proceed(CarA) is TRUE

(c) Two Simultaneous Emergency Vehicles

The current knowledge base can handle more than one emergency vehicle because the rules use:

∀x

This means the rules can be applied to every vehicle.

For example, add:

Vehicle(FireTruck)

EmergencyType(FireTruck)

Rule 1 gives:

ClearPath(FireTruck)

Rule 2 gives:

GreenSignal(FireTruck)

Therefore, the system can identify both:

ClearPath(Ambulance)

ClearPath(FireTruck)

and:

GreenSignal(Ambulance)

GreenSignal(FireTruck)

However, the current knowledge base is not sufficient for complete simultaneous

Additional Facts Required

For a second emergency vehicle:

Vehicle(FireTruck)

EmergencyType(FireTruck)

Location information can also be added:

At(Ambulance,Intersection1)

At(FireTruck,Intersection2)

Additional Rule 1: Priority

EmergencyType(x) AND HigherPriority(x,y)

→ PriorityVehicle(x)

Additional Rule 2: Conflict

PriorityVehicle(x) AND Conflicts(x,y)

→ Hold(y)

Additional Rule 3: Signal Control

GreenSignal(x) AND Conflicts(x,y)

→ NOT GreenSignal(y)

Additional Rule 4: Arrival Priority

EmergencyType(x) AND EmergencyType(y) AND EarlierArrival(x,y)

→ PriorityVehicle(x)

Case Study 3 — Agricultural Expert Reasoning System

An agricultural advisory logical agent operates with the following propositional knowledge base (KB):

1) P1: SoilDry → IrrigationNeeded

2) P2: IrrigationNeeded ∧ CropWheat → ApplyDripMethod

3) P3: ¬ApplyDripMethod ∧ CropWheat → CropAtRisk

4) Facts: SoilDry = True, CropWheat = True

(a) Convert all sentences P1, P2, P3 and the facts into Conjunctive Normal Form (CNF). Show each Conversion step (eliminate implication → move ¬ inward → distribute).

(b) Using Resolution Refutation, prove the theorem 'ApplyDripMethod'. Negate the goal, combine with the KB in CNF, and derive the empty clause □. Draw the resolution proof tree.

(c) If the fact 'SoilDry' is removed from the KB, analyze step-by-step how logical conclusions change. Does 'CropAtRisk' still follow? Apply appropriate reasoning patterns to justify.

Given Knowledge Base

P1: SoilDry → IrrigationNeeded

P2: IrrigationNeeded AND CropWheat → ApplyDripMethod

P3: NOT ApplyDripMethod AND CropWheat → CropAtRisk

Facts:

SoilDry = True

CropWheat = True

(a) Convert P1, P2, P3 and Facts into CNF

P1

Original:

SoilDry → IrrigationNeeded

Eliminate implication:

NOT SoilDry OR IrrigationNeeded

CNF:

C1 = NOT SoilDry OR IrrigationNeeded

P2

Original:

IrrigationNeeded AND CropWheat → ApplyDripMethod

Eliminate implication:

NOT (IrrigationNeeded AND CropWheat) OR ApplyDripMethod

Move NOT inward:

NOT IrrigationNeeded OR NOT CropWheat OR ApplyDripMethod

CNF:

C2 = NOT IrrigationNeeded OR NOT CropWheat OR ApplyDripMethod

P3

Original:

NOT ApplyDripMethod AND CropWheat → CropAtRisk

Eliminate implication:

NOT (NOT ApplyDripMethod AND CropWheat) OR CropAtRisk

Move NOT inward:

ApplyDripMethod OR NOT CropWheat OR CropAtRisk

CNF:

C3 = ApplyDripMethod OR NOT CropWheat OR CropAtRis

Fact 1

SoilDry = True

CNF:

C4 = SoilDry

Fact 2

CropWheat = True

CNF:

C5 = CropWheat

Final CNF Knowledge Base

C1 = NOT SoilDry OR IrrigationNeeded

C2 = NOT IrrigationNeeded OR NOT CropWheat OR ApplyDripMethod

C3 = ApplyDripMethod OR NOT CropWheat OR CropAtRisk

C4 = SoilDry

C5 = CropWheat

(b) Resolution Refutation

Goal

ApplyDripMethod

Negate the goal:

NOT ApplyDripMethod

Add it to the KB:

C6 = NOT ApplyDripMethod

Resolution Step 1

Take:

C1 = NOT SoilDry OR IrrigationNeeded

Take:

C4 = SoilDry

Resolve:

NOT SoilDry

with:

SoilDry

Result:

C7 = IrrigationNeeded

Resolution Step 2

Take:

C2 = NOT IrrigationNeeded OR NOT CropWheat OR ApplyDripMethod

Take:

C5 = CropWheat

Resolve:

NOT CropWheat

with:

CropWheat

Result:

C8 = NOT IrrigationNeeded OR ApplyDripMethod

Resolution Step 3

Take:

C8 = NOT IrrigationNeeded OR ApplyDripMethod

Take:

C7 = IrrigationNeeded

Resolve:

NOT IrrigationNeeded

with:

IrrigationNeeded

Result:

C9 = ApplyDripMethod

Resolution Step 4

Take:

C9 = ApplyDripMethod

Take:

C6 = NOT ApplyDripMethod

Resolve:

ApplyDripMethod

with:

NOT ApplyDripMethod

Result:

C10 = □

Resolution Proof Tree

C4

SoilDry

|

|

C1

NOT SoilDry OR IrrigationNeeded

|

↓

C7

IrrigationNeeded

|

|

C2

NOT IrrigationNeeded OR NOT CropWheat OR ApplyDripMethod

|

C5

CropWheat

|

↓

C8

NOT IrrigationNeeded OR ApplyDripMethod

|

|

C7

IrrigationNeeded

|

↓

C9

ApplyDripMethod

|

|

C6

NOT ApplyDripMethod

(c) Remove SoilDry from the KB

Original fact:

SoilDry = True

Remove this fact.

Therefore:

C4 = SoilDry

is removed.

Remaining facts:

CropWheat = True

Step 1: Check IrrigationNeeded

Rule:

SoilDry → IrrigationNeeded

CNF:

NOT SoilDry OR IrrigationNeeded

We no longer have:

SoilDry

Therefore:

IrrigationNeeded cannot be proved.

Result:

IrrigationNeeded = Unknown

Step 2: Check ApplyDripMethod

Rule:

IrrigationNeeded AND CropWheat → ApplyDripMethod

We have:

CropWheat = True

But we do not have:

IrrigationNeeded = True

Therefore:

ApplyDripMethod cannot be proved.

Result:

ApplyDripMethod = Unknown

Step 3: Check CropAtRisk

Rule:

NOT ApplyDripMethod AND CropWheat → CropAtRisk

We know:

CropWheat = True

But we do not know:

NOT ApplyDripMethod

Important:

ApplyDripMethod = Unknown

does NOT mean:

NOT ApplyDripMethod = True

Therefore:

CropAtRisk cannot be proved.

Final Answer

When SoilDry is present:

SoilDry

↓

IrrigationNeeded

↓

ApplyDripMethod

When SoilDry is removed:

Case Study 4 — Wumpus World Knowledge Agent

An AI agent navigating the Wumpus World perceives: Stench at [1,2], Breeze at [1,1], Glitter at [2,2]. The

agent uses a propositional KB with these rules:

1) Stench [1,2] → Wumpus is adjacent to cell [1,2]

2) Breeze [1,1] → Pit is adjacent to cell [1,1]

3) Glitter [x,y] → Gold is at [x,y]

4) ¬Wumpus [x,y] ∧ ¬Pit[x,y] → Safe[x,y]

(a) Construct the agent's belief state from its percepts. Apply Modus Ponens systematically to derive new knowledge facts. List all derived conclusions.

(b) Using Forward Chaining from the agent's percepts, identify the possible locations of the Wumpus and Pit. Show each reasoning step precisely.

(c) Evaluate whether the path [1,1] → [1,2] → [2,2] is safe for the agent to collect the gold. Use the knowledge derived in (a) and (b) to justify your conclusion with logical inference.

(a) Construct the Belief State

Step 1: Initial Percepts

The agent perceives:

Stench[1,2] = True

Breeze[1,1] = True

Glitter[2,2] = True

Therefore, the initial belief state is:

Belief State = {Stench[1,2], Breeze[1,1], Glitter[2,2]}

Step 2: Apply Rule 1

Rule:

Stench[1,2] → Wumpus adjacent to [1,2]

Known fact:

Stench[1,2]

Therefore, by Modus Ponens:

Wumpus is adjacent to [1,2]

The adjacent cells of [1,2] are:

[1,1]

[1,3]

[2,2]

Therefore:

Wumpus ∈ {[1,1], [1,3], [2,2]}

Step 3: Apply Rule 2

Rule:

Breeze[1,1] → Pit adjacent to [1,1]

Known fact:

Breeze[1,1]

Therefore, by Modus Ponens:

Pit is adjacent to [1,1]

The adjacent cells of [1,1] are:

[1,2]

[2,1]

Therefore:

Pit ∈ {[1,2], [2,1]}

Step 4: Apply Rule 3

Rule:

Glitter[x,y] → Gold[x,y]

Known fact:

Glitter[2,2]

Therefore:

Gold[2,2]

So the agent knows that gold is located at:

[2,2]

Step 5: Rule 4

Rule:

NOT Wumpus[x,y] AND NOT Pit[x,y] → Safe[x,y]

At this stage, the agent has not established that any particular cell is both:

NOT Wumpus

and:

NOT Pit

Therefore, Rule 4 cannot yet be applied to any specific cell.

Derived Conclusions

1. Stench[1,2]
2. Breeze[1,1]
3. Glitter[2,2]
4. Wumpus is adjacent to [1,2]

(b) Forward Chaining

Step 1

Initial fact:

Stench[1,2]

Apply Rule 1:

Stench[1,2] → Wumpus adjacent to [1,2]

Step 2

Initial fact:

Breeze[1,1]

Apply Rule 2:

Breeze[1,1] → Pit adjacent to [1,1]

Conclusion:

Pit ∈ {[1,2], [2,1]}

Step 3

Initial fact:

Glitter[2,2]

Apply Rule 3:

Glitter[2,2] → Gold[2,2]

Conclusion:

Gold[2,2]

Possible Wumpus Locations

From:

Stench[1,2]

The Wumpus can be in:

[1,1]

[1,3]

[2,2]

Therefore:

Possible Wumpus locations = {[1,1], [1,3], [2,2]}

Possible Pit Locations

From:

Breeze[1,1]

The Pit can be in:

[1,2]

[2,1]

Therefore:

Possible Pit locations = {[1,2], [2,1]}

Forward Chaining Summary

Stench[1,2]

↓

Wumpus adjacent to [1,2]

↓

Wumpus ∈ {[1,1], [1,3], [2,2]}

Breeze[1,1]

↓

Pit adjacent to [1,1]

↓

Pit ∈ {[1,2], [2,1]}

Glitter[2,2]

↓

Gold[2,2]

(c) Safety of Path

Proposed path:

[1,1] → [1,2] → [2,2]

Goal:

Reach [2,2] and collect Gold

We know:

Gold[2,2]

Therefore:

[2,2] contains Gold

However, we must check whether each cell is safe.

Check Cell [1,1]

The agent starts at:

[1,1]

We know:

Breeze[1,1]

This indicates:

Pit ∈ {[1,2], [2,1]}

It does not prove that:

Pit[1,1]

Therefore:

[1,1] is not identified as a Pit.

Also, the current knowledge does not establish:

Wumpus[1,1]

So:

[1,1] is not proven unsafe.

Check Cell [1,2]

We know:

Stench[1,2]

Therefore:

Wumpus ∈ {[1,1], [1,3], [2,2]}

The Stench does not mean that:

Wumpus[1,2]

It means that the Wumpus is adjacent to [1,2].

For the Pit:

Pit ∈ {[1,2], [2,1]}

Therefore:

[1,2] may contain a Pit.

Hence:

[1,2] is NOT proven safe.

Check Cell [2,2]

We know:

Gold[2,2]

But we also know from the Stench:

Wumpus ∈ {[1,1], [1,3], [2,2]}

Therefore:

[2,2] may contain the Wumpus.

The Pit possibilities are:

Pit ∈ {[1,2], [2,1]}

Therefore, the Pit is not currently known to be at [2,2].

However:

[2,2] may contain the Wumpus.

Therefore:

[2,2] is NOT proven safe.

Path Safety Table

Cell	Knowledge	Safety
[1,1]	No Wumpus/Pit confirmed	Not proven unsafe
[1,2]	May contain Pit	Not safe to assume
[2,2]	May contain Wumpus	Not safe to assume

The agent should not assume the path is safe until additional percepts or knowledge eliminate these possible hazards.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand